Number system

Page No.
Date

Every computer stores numbers, letters, and other special characters in coded form.

Number systems are of two types - non-positional and positional.

Non-positional number system.
In early days, human beings counted on fingers. When counting beyond ten figures, they used stones, pebbles or sticks to indicate values. This method of counting uses an additive approach or non-positional number system. In this, we have symbols such as I for 1, II for 2, III for 3, IIII for 4, IIIII for 5 and so on.
to find value, one has to count no. of symbol.
Disadvantage: It is very difficult to perform arithmetic.

Positional number system. (binary, octal, hexadecimal)
In Positional number system, there are few symbols called digits. The value of each digit in such a number is determined by three considerations:

(i) The digit itself
(ii) position of digit in number
(iii) The base of number system (where base is defined as total number of digits available in number system)

The value of base in all positional number system suggest following characteristics:

(i) value of base determines total no. of different symbols or digits available in number system. The first one is always 0.
(ii) Maximum value of single digit is always equal to one less than value of base.

eg: base 10 → (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
↑ one less than base

Decimal number system → base 10. (0 to 9)
Binary number system → base 2 (0 & 1)
octal number system → base 8 (0 to 7)
hexadecimal number system → base 16 (0 to 15)

Short form of binary digit is "bit"

An n-bit number in a binary number consist of 'n' bits.
ex: 3 bit number consist of 3 bits

Any decimal number in range 0 to $2^n - 1$ can be represented in binary form as n-bit number

ex: 3bit number ranges from 0 to $2^3 - 1 = 2^3 - 1 = 7$
ie 0 to 7

There are several occasions when computer professional need to know about raw data contained in computer's memory. A commonly used way of doing this is to print memory contents on printer. This printout is called memory dump.
M.D which are in binary number, would have many pages 0s and 1s.

converting from one number system to another.

(i) converting from another base to decimal

( )$_{2/8/16}$ ⟶ ( )$_{10}$

ex: $(11001)_2 \longrightarrow (\quad)_{10}$

$= 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$

$= 16 + 8 + 0 + 0 + 1$

$= (25)_{10}$

ex: $(1AC)_{16} \longrightarrow (\quad)_{10}$

$= 1 \times 16^2 + A \times 16^1 + C \times 16^0$

$= 1 \times 256 + 10 \times 16 + 12 \times 1$

$= (428)_{10}$

ex: $(4052)_7 \longrightarrow (\quad)_{10}$

if any other base is asked in exam.

$4 \times 7^3 + 0 \times 7^2 + 5 \times 7^1 + 2 \times 7^0$

$= 1372 + 0 + 35 + 2$

$= (1409)_{10}$

(ii) converting from decimal to another base.

( )$_{10}$ ⟶ ( )$_{2/8/16}$

ex: $(25)_{10} \longrightarrow (\quad)_2$

2	25	
2	12	→ 1
2	6	→ 0
2	3	→ 0
2	1	→ 1
	0	→ 1

$(11001)_2$

ex: $(952)_{10} \longrightarrow (\quad)_8$

8	952	
8	119	→ 0
8	14	→ 7
8	1	→ 6
	0	→ 1

$= (1670)_8$

ex: $(1715)_{10} \longrightarrow (\quad)_{12}$

if any other base is asked.

12	1715	
12	142	→ 11 = B
12	11	→ 10 = A
	0	→ 11 = B

$= (BAB)_{12}$

(iii) converting from a base other than 10 to another base other than 10.

ex: $(545)_6 \longrightarrow (\quad)_4$

step 1: $(545)_6 \longrightarrow (\quad)_{10}$

first convert base 6 to base 10 and from base 10 to base 4.

$= 5 \times 6^2 + 4 \times 6^1 + 5 \times 6^0$

$= 180 + 24 + 5$

$= (209)_{10}$

step 2: $(209)_{10} \longrightarrow (\quad)_4$

4	209	
4	52	1
4	13	0
4	3	1
	0	3

$= (3101)_4$

ie: $(545)_6 \rightarrow (3101)_4$

Page No.
Date

(iv) Shortcut method for binary to octal conversion.

ex: $(101110)_2 \longrightarrow (\quad)_8$

divide binary digit into group of three.
convert each group into one octal digit

101110

↓

101 110 $= (56)_8$

5 6

ex: $(1101010)_2 \longrightarrow (\quad)_8$

001 101 010 $= (152)_8$

add 2 zero ↓ 1 5 2

(v) Shortcut method for octal to binary conversion.

ex: $(6751)_8 \longrightarrow (\quad)_2$ reverse of above.

6 7 5 1

↓ ↓ ↓ ↓

110 111 101 001

$= (110111101001)_2$

(vi) Shortcut method for binary to hexadecimal conversion.

divide binary digit into group of four.
Convert each group into one hexadecimal digit

ex: $(2AB)_{16} \rightarrow (\quad)_2$

2 A B

0010 1010 1011

$= (001010101011)_2$

(vii) Shortcut method for hexadecimal to binary conversion

ex: $(ABC)_{16} \rightarrow (\quad)_2$

A B C

1010 1011 1100

$= (101010111100)_2$

Fractional numbers

In binary number system, fractional numbers are formed in the same way as in decimal number system.

In decimal number system,

ex: $(0.235) = 2\times10^{-1} + 3\times10^{-2} + 5\times10^{-3}$

ex: $(68.53) = 6\times10^{1} + 8\times10^{0} + 5\times10^{-1} + 3\times10^{-2}$

In binary number system,

ex: $(0.101) = 1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3}$

ex: $(10.01) = 1\times2^{1} + 0\times2^{0} + 0\times2^{-1} + 1\times2^{-2}$

Position	4	3	2	1	0	.	−1	−2	−3	−4
Positionvalue	2^4	2^3	2^2	2^1	2^0		2^{-1}	2^{-2}	2^{-3}	2^{-4}
Quantity Represented	16	8	4	2	1		$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{16}$

Page No.
Date

ex: Find decimal equivalent of binary number 110.101

Sol: $(110.101)_2$

$= 1\times2^2 + 1\times2^1 + 0\times2^0 + 1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3}$

$= 4 + 2 + 0 + 0.5 + 0 + 0.125$

$= 6 + 0.5 + 0.125$

$= (6.625)_{10}$

ex: Find decimal equivalent of the octal number 127.54

Sol: $(127.54)_8$

$= 1\times8^2 + 2\times8^1 + 7\times8^0 + 5\times8^{-1} + 4\times8^{-2}$

$= 64 + 16 + 7 + 5/8 + 4/64$

$= 87 + 0.625 + 0.0625$

$= (87.6875)_{10}$

ex: Find decimal equivalent of hexadecimal number 2B.C4.

Sol: $(2B.C4)_{16} = 2\times16^1 + B\times16^0 + C\times16^{-1} + 4\times16^{-2}$

$= 32 + B + C/16 + 4/256$

$= 32 + 11 + 0.75 + 0.015625$

$= (43.765652)_{10}$

maximum value of single digit of octal (7) / hexadecimal (15) is equal to maximum value of three/four digit in binary.

ex: maximum value of octal = 7 → needs three bit to represent

maximum value of hexadecimal = 15 (F) → needs four digit to represent

maximum value of three digit = 111 = 7

maximum value of four digit = 1111 = 15 (F)

Character	EBCDIC Code		Hexadecimal Equivalent
	Zone	Digit	
A	1100	0001	C1
B	1100	0010	C2
C	1100	0011	C3
D	1100	0100	C4
E	1100	0101	C5
F	1100	0110	C6
G	1100	0111	C7
H	1100	1000	C8
I	1100	1001	C9
J	1101	0001	D1
K	1101	0010	D2
L	1101	0011	D3
M	1101	0100	D4
N	1101	0101	D5
O	1101	0110	D6
P	1101	0111	D7
Q	1101	1000	D8
R	1101	1001	D9

Character	EBCDIC Code		Hexadecimal Equivalent
	Zone	Digit	
S	1110	0010	E2
T	1110	0011	E3
U	1110	0100	E4
V	1110	0101	E5
W	1110	0110	E6
X	1110	0111	E7
Y	1110	1000	E8
Z	1110	1001	E9
0	1111	0000	F0
1	1111	0001	F1
2	1111	0010	F2
3	1111	0011	F3
4	1111	0100	F4
5	1111	0101	F5
6	1111	0110	F6
7	1111	0111	F7
8	1111	1000	F8
9	1111	1001	F9

Decimal	Hexadecimal	Binary	Octal
0	0	0	0
1	1	1	1
2	2	10	2
3	3	11	3
4	4	100	4
5	5	101	5
6	6	110	6
7	7	111	7
8	8	1000	10
9	9	1001	11
10	A	1010	12
11	B	1011	13
12	C	1100	14
13	D	1101	15
14	E	1110	16
15	F	1111	17
16	10	10000	20

computer codes

Page No.
Date

Numeric data is not only form of data handled by a computer. We often require to process alphanumeric data also.
An alphanumeric data is string of symbols, where symbol may be letters A, B, C, ..., Z or one of digits 0, 1, 2, ... 9 or special characters such as + - * / , . ( ) = (space or blank) etc.
An alphabetic data consists A, B, C ... Z and blank character.
Numeric data consist of 0, 1, 2, ... 9 only.

Bits 0, 1 represent data internally. computer uses binary coding schemes to represent data internally.
Group of bits used to represent symbol is called byte.

a byte is referred to as "n-bit byte" → byte means 8-bit byte because most computer use 8 bit to represent a symbol.

BCD (Binary Coded Decimal) → 4 bits or 6 bits
one of the early computer codes.
↳ converts each digit of decimal number into its binary equivalent instead of converting entire decimal value into binary number.

since 8 and 9 requires 4 bits, all decimal digit are represented in BCD by 4 bits

ex: $(42)_{10} = \underbrace{0100}_{4}\ \underbrace{0010}_{2}$ or 01000010 in BCD

4 bits means $2^4 = 16$ combinations are possible but for decimal (0 to 9) only first 10 combinations are used to represent decimal digit. Rest are not used in BCD.

Page No.
Date

Computer designers commonly use six bits to represent characters in BCD code.

↳ four BCD numeric place position are retained but two additional zone position are added.

six bits → 2^6 = 64 different combinations are possible. This is sufficient to code decimal digit (10), alphanumeric letters (26) and other special characters (28).

Character	BCD Code		Octal Equivalent
	Zone	Digit	
A	11	0001	61
B	11	0010	62
C	11	0011	63
D	11	0100	64
E	11	0101	65
F	11	0110	66
G	11	0111	67
H	11	1000	70
I	11	1001	71
J	10	0001	41
K	10	0010	42
L	10	0011	43
M	10	0100	44
N	10	0101	45
O	10	0110	46
P	10	0111	47
Q	10	1000	50
R	10	1001	51

Character	BCD Code		Octal Equivalent
	Zone	Digit	
S	01	0010	22
T	01	0011	23
U	01	0100	24
V	01	0101	25
W	01	0110	26
X	01	0111	27
Y	01	1000	30
Z	01	1001	31
0	00	0000	00
1	00	0001	01
2	00	0010	02
3	00	0011	03
4	00	0100	04
5	00	0101	05
6	00	0110	06
7	00	0111	07
8	00	1000	10
9	00	1001	11

ex: Write binary digits used to record BASE in BCD.

Sol: B = 110010 S = 010010

A = 110001 E = 110101

Binary digit = 110010 110001 010010 110101

B A S E

character	octal equivalent	zone	always miss 08, 09
0-9	0-11	00	28, 29
S-Z	22-31	01	48, 49
J-R	41-51	10	68, 69
A-I	61-71	11	

How to remember

Page No.
Date

EBCDIC (Extended Binary-Coded Decimal Interchange code) → 8 bit

Major problem with BCD code is that it can represent only $2^6 = 64$ characters. This is not sufficient to represent decimal number (10), lowercase (small) letters (26), uppercase (capital) letters (26) and large no. of other special characters (28+).

BCD code extended from 6 bits to 8 bits is known as EBCDIC. 2 bits were added in additional zone.
Now $2^8 = 256$ different characters can be represented.

In addition to various character requirement mentioned above, this also allows a large variety of printable characters and several non-printable control characters control characters are used to control such activities as printer vertical spacing, movement of cursor on terminal etc.

EBCDIC is 8 bit code, it can be easily divided into two 4-bit group.

4 bit groups can be represented by one hexadecimal digit

Developed by IBM, EBCDIC code is used in most IBM models, and in many other computers.

Page No.
Date

Zoned and Packed Decimal Numbers.

Numeric values needs special consideration because number can be positive, negative or unsigned (implies positive).

Here, when a numeric value is represented in EBCDIC, a sign indicator is used in zone position of rightmost digit.

A sign indicator of hexadecimal C is a plus sign, hexadecimal D is a minus sign, hexadecimal F means number is unsigned.

Since each decimal digit has a zone with it, number coded in EBCDIC are called zoned decimal numbers.

Printers can print only those numeric character that are zoned-decimal format.

Most computers cannot perform arithmetic operations on zoned-decimal data. Here, before any arithmetic operations can be performed data must be converted to format on which arithmetic operation can be performed. one such acceptable format is packed decimal format.

steps to convert zoned decimal format to packed decimal format.

step 1: The zone half and digit half of rightmost byte are reversed. This moves sign to the extreme right of number.

step 2: All remaining zones are dropped out.

packed data requires less no of bytes (group of 8 bits) as compared to zoned data.

Page No.
Date

Numeric EBCDIC Sign
Value Indicator

Numeric Value	EBCDIC	Sign Indicator
345	F3F4F5	F for unsigned
+345	F3F4C5	C for positive
-345	F3F4D5	D for negative

Numeric Value	Zoned Format	Packed Format
345	F3F4F5	345F
+345	F3F4C5	345C
-345	F3F4D5	345D
3456	F3F4F5F6	03456F

ex: Write EBCDIC coding for word "BIT" in both binary and hexadecimal notations. How many bytes are required to store this word using this coding?

In EBCDIC
B = 11000010 in binary and C2 in hexadecimal
I = 11001001 in binary and C9 in hexadecimal
T = 11100011 in binary and E3 in hexadecimal

$\underbrace{11000010}_{B}\ \underbrace{11001001}_{I}\ \underbrace{11100011}_{T}$ in binary

$\underbrace{C2}_{B}\ \underbrace{C9}_{I}\ \underbrace{E3}_{T}$ in hexadecimal.

In EBCDIC, each character require one byte.
BIT has 3 character, hence requires three byte.
or each hexadecimal require 4 bits, total 6 digits (C2C9E3), hence 4X6= 24 bits or 3 byte.
ASCII (American standard code for Information Interchange).
↳ is used almost exclusively to represent data internally in microcomputers.
↳ American National Standards Institute (ANSI) published ASCII standard in 1963. and after changes finally in 1986 brought it in its present form.

zone and packed decimal example.

Page No. | Date

Ex: Write EBCDIC in zoned decimal coding for value +256 (use hexadecimal). How many bytes are required for this representation.

Sol: +256 = F2F5C6 in EBCDIC (hexadecimal)
↳ (positive)

Each hexadecimal digit requires 4 bits.
There are altogether 6 hexadecimal digit (F2F5C6).
Therefore 6 X 4 = 24 bits or 3 bytes.

Ex: Write −128 as packed decimal number (use hexadecimal). How many bytes are required for this representation?

↳ negative
−128 = F1F2D8 in EBCDIC
= 128D in packed format

Each hexadecimal require 4 bits
no. of digits in packed format = 4 (128D)
Therefore 4 X 4 = 16 bits or 2 bytes.

ASCII (continued)

∟ ASCII-7 : 7 bit code that represent $2^7 = 128$ different characters. computer using 8-bit byte (group of 8 bits or 1 byte) and the 7-bit ASCII either set the 8th bit (leftmost bit) of each byte as zero or use it as a parity bit.

∟ ASCII-8 : It is an 8-bit code that can represent $2^8 = 256$ different characters. The additional bit is added to left of 7th bit (leftmost bit) of ASCII-7 codes.

Page No.
Date

ex: Write ASCII-7 coding for word "GIRL" in both binary and hexadecimal notation. How many bytes are required to store this word using this coding?

Sol: G = 100 0111 in binary and 47 in hexadecimal
I = 100 1001 in binary and 49 in hexadecimal
R = 101 0010 in binary and 52 in hexadecimal
L = 100 1100 in binary and 4C in hexadecimal

1000111 1001001 1010010 1001100 in binary
(G) (I) (R) (L)

47 49 52 4C in hexadecimal
(G) (I) (R) (L)

four characters, hence 4 bytes.

ex: Write ASCII-8 coding for word John in both binary and hexadecimal. How many bytes are required to store this word using this coding?

J = 0100 1010 in binary 4A in hexadecimal
o = 0100 1110 in binary 6F in hexadecimal
h = 0100 1000 in binary 68 in hexadecimal
n = 0100 1110 in binary 6E in hexadecimal

0100 1010 0100 1110 01001000 01001110 in binary
(J) (o) (h) (n)

4A 6F 68 6E in hexadecimal.
(J) (o) (h) (n)

four character, 4 bytes.

Page No.
Date

The codes of first 128 characters are identical in ASCII-7 & ASCII-8.

ASCII also uses hexadecimal as its four-to-one shortcut notation for memory dump.

Excess-3 code. (XS-3 code)
↳ are derived from 8421 BCD code words adding $(0011)_2$ or $(3)_{10}$ to each code word in 8421.

Decimal Number → 8421 BCD $\xrightarrow[0011]{\text{Add}}$ Excess-3

Gray code. (Unit distance code) (cyclic code)
↳ only one bit will change each time the decimal number is incremented. As only one bit changes at a time, gray code is called as a unit distance code.
It is also called cyclic code.

Decimal	BCD8421	Excess-3 (BCD + 0011)	Gray
0	0000	0011	0000
1	0001	0100	0001
2	0010	0101	0011
3	0011	0110	0010
4	0100	0111	0110
5	0101	1000	0111
6	0110	1001	0101
7	0111	1010	0100
8	1000	1011	1100
9	1001	1100	1101

Page No.
Date

Unicode : " provides a unique number for every character, no matter what the platform, no matter what the program, no matter what the language".

Unicode is internationally accepted as a universal character-encoding standard because:

(i) It provides a consistent way of encoding multilingual plain text.

(ii) It defines codes for character in all major languages of the world

(iii) It defines codes for special characters, mathematical symbols, technical symbol and diacritics. Diacritics are modifying character mark such as tilde (~)

(iv) can encode as many as million characters.

(v) It reserves a part of code space for private use to enable users to assign codes for their own character & symbols.

(vi) It affords simplicity and consistency of ASCII

(vii) It specifies an algorithm for presentation of text with bi-directional behaviour.

Unicode Encoding Forms.

In addition to defining the identity of each character and its numeric value (also known as code point), character encoding standards also define internal representation (also known as encoding forms) of each character. Unicode defines three encoding forms:

(i) UTF - 8 (Unicode transformation Format - 8)

(ii) UTF - 16

(iii) UTF - 32

UTF-8	UTF-16	UTF-32
→ 1 byte = 8 bit	→ 1 word = 16 bits	→ 1 word = 16 bits
→ byte oriented (designed entirely around ASCII and good for HTML)	→ one word or 2 words → word oriented	→ double word (32 bits) → double word oriented
→ variable length encoding	→ variable length encoding	→ fixed length encoding of two words
→ 1, 2, 3 or 4 bytes		→ double word (32 bit code) encodes each character. → useful where memory space is not a concern but fixed width access to character is desired

At most 4 bytes (32 bits) are required for each character in all three forms of encoding

Unicode standard also provides the facility to transform Unicode encoded data from one form to another without loss of data.

Collating Sequence : ordering of symbols in an encoding standard. mostly case sensitive dictionary collating sequence is adopted.

Page No.
Date

Following rules are observed:

1. Letters are considered in alphabetic order (A<B<C....<Z) and (a<b<c....<z)
2. Digits are considered in numeric order (0<1<...<9)

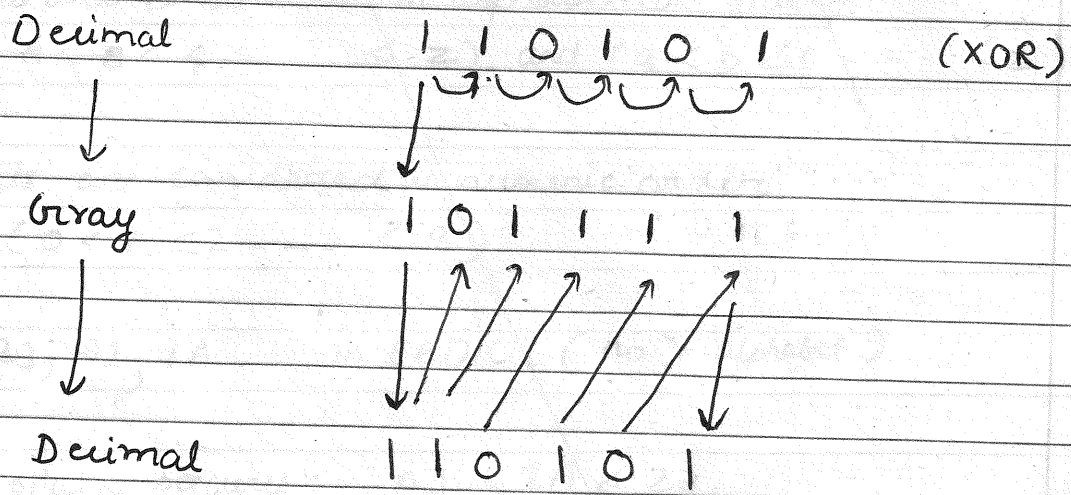
In EBCDIC, numeric characters have greater values than alphabetic characters
ex: 23, A1, 1A
will be sorted as A1 < 1A < 23.

In ASCII, alphabetic characters have greater than number characters and lowercase letter have values greater than upper case letter.
Hence, lowercase > uppercase > numbers.

eg: 23, A1, 1A, a2, 2a, aA, Aa

Sorted order will be
1A < 23 < 2a < A1 < Aa < a2 < aA

eg: Convert decimal ⟶ gray code ⟶ decimal



→ also called 8421 code.

Note: BCD → is a 6 bit code that can represent 64 different character.

EBCDIC → (Extended Binary Coded Decimal Interchange code) is a 8 bit code that can represent 256 different character. without parity. (used in large comp). 9th bit can be used for parity.

ASCII → (American standard Code for Information Interchange)

ASCII-7 is a 7 bit code that can represent 128 different character

ASCII-8 is a 8 bit code that can represent 256 " " .
(used in small comp)

Unicode → consistent way of encoding multilingual plain text (different languages). It has to capacity to encode as many as million character.

UTF-8 UTF-16 UTF-32

Collating sequence → ordering of symbol in an encoding standard

→ also called cyclic code or reflective code, minimum error code bcz error is minimum

Gray code: is a binary code. used ~~as~~ in shaft encoder.
Bits are arranged in such a way that only one bit changes at a time. It's use reduces error in reading shaft position. The largest possible error will be one least significant bit.

0	0000	2	0011
1	0001	3	0010

and so on.

→ is a 4 bit code, also called self complementing code

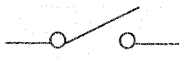
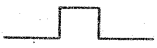
Excess-3 code: to overcome disadvantage of BCD in forming complement. This code is formed by adding 3 to decimal number and then forming binary coded number

eg: Excess-3 code of 5 = 5+3 = 8 → 1000

So, Excess 3 code of 5 is 1000.

computer arithmetic

Page No.
Date

Binary State	ON (1)	OFF (0)
Bulb		
Switch		
Circuit Pulse		

Binary arithmetic → addition, subtraction, multiplication & division.

Addition.

0 + 0 = 0

0 + 1 = 1

1 + 0 = 1

1 + 1 = 0 plus a carry of 1 to next higher column.

ex: add 101 and 10 in both decimal and binary forms.

Binary	Decimal
101	5
+ 10	+ 2
111	7

ex: add binary number 10011 and 1001 in both decimal & binary forms.

Binary	Decimal
(1)(1)	
10011	19
1001	+ 9
11100	28

Page No.
Date

ex: add 100111 and 11011 in decimal & binary

binary

```
 (1)(1)(1)(1)(1)
  100111          39      decimal
+ 11011         + 27
--------        ----
 1000010         66
```

Subtraction

If subtrahend (lower digit) is larger than minuend (the upper digit), it is necessary to borrow from column to left.

In decimal, 10 is borrowed, In binary 2 is borrowed, In octal 8 is borrowed, In hexadecimal 16 is borrowed.

$0 - 0 = 0$

$0 - 1 = 1$ with a borrow from next column

$1 - 0 = 1$

$1 - 1 = 0$

ex: subtract 01110_2 from 10101_2

```
    1 2
  0 ~ 0 2
  ~0 ~0 1
- 0 1 1 1 0
-----------
  0 0 1 1 1
```

ex: subtract 0111000_2 from 1011100_2

```
    0 2
  ~0 1 1 1 0 0
- 0 1 1 1 0 0 0
---------------
    1 0 0 1 0 0
```

Page No.
Date

additive method of subtraction (complementary subtraction)
complement is defined as difference between number and base raised to n^{th} power minus one. ie $(Base)^n - 1$.

step 1: Find the complement of number you are subtracting (subtrahend)
step 2: add this to number from which you are taking away (minuend)
step 3: If there is carry of 1, add it to obtain result, if there is no carry, recomplement sum and attach a negative sign to obtain result.

ex: Find 56_{10}, 10101_2, 92_{10}, 6_8 complement

Complement of $56_{10} \Rightarrow 10^2 - 1 - 56$
$= 99 - 56$
$= 43_{10}$

complement of $92_{10} \Rightarrow 10^2 - 1 - 92$
$= 99 - 92$
$= 7_{10}$

complement of $10101_2 \Rightarrow (2)^5 - 1 = 31_{10} = 11111_2$
$= 11111 - 10101$
$= 01010_2$

complement of $6_8 \Rightarrow (8)^1 - 1 = 7_8$
$= 7_8 - 6_8$
$= 1_8$

Page No.
Date

ex: subtract 56_{10} from 92_{10} using complementary method.

step 1: 56_{10} complement

$= 10^2 - 1 - 56$

$= 99 - 56$

$= 43_{10}$

verify $92_{10} - 56_{10}$

$= 36_{10}$

step 2:
$$\begin{array}{r} 92 \\ +\ 43 \\ \hline (1)35 \\ \hookrightarrow +1 \\ \hline 36 \end{array}$$

carry ← (1)

result: 36

ex: subtract 35_{10} from 18_{10} using complementary method.

step 1: complement of 35_{10}

$= 10^2 - 1 - 35$

$= 99 - 35$

$= 64_{10}$

verify
$$\begin{array}{r} 18 \\ -\ 35 \\ \hline -17 \end{array}$$

step 2:
$$\begin{array}{r} 18 \\ +\ 64 \\ \hline 82 \end{array}$$
no carry

step 3: recomplement sum and attach a negative sign

result $= -(99 - 82)$

$= -17$